**Projet de Programmation**
**Master Informatique**
**2025–2026**

université de BORDEAUX

# Carcassonne – Python

**Rapport préliminaire**

Anthony BEAUVIEUX, Tony HUANG, Maxence SALIN, Clément SIMON, Loïc THIN

## 1 Présentation du jeu

### 1.1 Contexte

**Carcassonne** est un jeu de stratégie et de placement de tuiles créé par Klaus-Jürgen Wrede en 2000, conçu pour 2 à 6 joueurs. Il s'agit d'un jeu stochastique à information parfaite, c'est-à-dire un jeu où le hasard intervient dans le tirage des tuiles mais où l'ensemble des éléments présents sur le plateau est visible de tous les joueurs.



FIGURE 1 – Tuile de départ

La version considérée ici utilise les 72 tuiles du jeu de base. Parmi ces tuiles, se trouve la tuile de départ (figure 1) qui est placée au centre du plateau avant le début de la partie. Cette tuile est la même pour chaque partie et comporte une *City* en haut (couleur marron), une *Road* qui traverse la tuile de gauche à droite et un *Field* en bas. Chaque joueur dispose de 7 pions de sa couleur pour marquer des points au cours de la partie. Le but du jeu est que les joueurs construisent progressivement le plateau, à tour de rôle, en posant des tuiles de manière cohérente et en plaçant stratégiquement leurs pions pour accumuler des points. Le vainqueur est le joueur qui totalise le plus de points à la fin de la partie.

### 1.2 Règles

Les règles de **Carcassonne** utilisées sont celles de la version 2012[3].

#### 1.2.1 Spécificités

**Tuile**

Chaque tuile possède quatre côtés ainsi qu'un centre. Chaque côté correspond à un type de terrain parmi *Road*, *City* ou *Field*. Le centre peut représenter une *City*, un *Field*, une *Banner* ou un *Monastery*.



FIGURE 2 – Exemple d'une tuile où des *Fields* possèdent un *Monastery*



FIGURE 3 – Exemple d'une tuile où une *City* possède une *Banner*

**Pion**

Lorsqu'un pion est placé sur une tuile, il prend un rôle en fonction du type de terrain occupé : *Knight* sur une *City*, *Farmer* sur un *Field*, *Thief* sur une *Road* et *Monk* sur un *Monastery*.

### 1.2.2 Déroulement d'un tour

Les joueurs jouent à tour de rôle selon un ordre prédéfini (généralement dans le sens horaire, en commençant par le joueur le plus jeune). Lors de son tour, un joueur effectue les actions suivantes dans l'ordre indiqué :

1. **Piocher une tuile :** Le joueur pioche une tuile dans la pioche actuellement active. Cette tuile est visible par l'ensemble des joueurs.
2. **Placer la tuile :** La tuile piochée doit être placée sur le plateau en respectant les règles suivantes :
   - La tuile doit être adjacente à au moins une autre tuile déjà posée.
   - Les côtés en contact doivent correspondre à des terrains de même type.
3. **Placer un pion (optionnel) :** Le joueur peut placer un pion sur un élément de terrain de la tuile qu'il vient de poser (*City*, *Road*, *Field*, *Monastery*).
4. **Restriction de placement des pions :** Si l'élément de terrain choisi est relié à un élément du même type déjà occupé par un pion, aucun nouveau pion ne peut y être placé. Les *Road* et *City* agissent comme des séparateurs entre les *Field*.
5. **Résolution des terrains complétés :** Si la pose de la tuile permet de compléter un ou plusieurs terrains, ceux-ci sont résolus immédiatement et les joueurs concernés marquent des points selon certaines règles. (voir partie 1.2.3)
6. **Retour des pions :** Tous les pions associés à des *City*, *Road* ou *Monastery* complétés durant le tour actuel sont retirés du plateau et rendus à leurs propriétaires. Les pions *Farmer* restent en jeu jusqu'à la fin de la partie.

### 1.2.3 Score

Les points sont attribués à deux moments :

- Pendant la partie : dès qu'un terrain (*City*, *Road*, *Monastery*) est complété (figure 4).
- À la fin de la partie : pour les terrains incomplets (*City*, *Road*, *Monastery*) ainsi que pour les *Field*.

Les joueurs ajoutent alors à leur score les points correspondant aux terrains selon les règles suivantes :

**Road :**

Une *Road* rapporte 1 point par tuile la composant au joueur possédant le plus de pions (*Thief*) sur cette *Road*. En cas d'égalité, tous les joueurs concernés marquent ces points.

**City :**

Une *City* complète rapporte 2 points par tuile qui la compose ainsi que 2 points par *Banner* présent, pour le joueur possédant le plus de pions (*Knight*). Si la *City* est incomplète à la fin de la partie, chaque tuile rapporte 1 point et chaque *Banner* rapporte également 1 point. En cas d'égalité, les points sont attribués à tous les joueurs concernés.

**Monastery :**

Un *Monastery* rapporte 1 point par tuile ayant un sommet ou côté en commun, y compris la tuile du *Monastery*. Le score maximal d'un *Monastery* est donc de 9 points.

**Field :**

Chaque ensemble de *Fields* connectés ou isolés est attribué au joueur possédant le plus de pions (*Farmer*). Il marque alors 3 points par *City* complète adjacente aux *Field*. En cas d'égalité,tous les joueurs concernés marquent ces points.

A l'issue du décompte final, le joueur ayant obtenu le plus grand nombre de points remporte la partie.

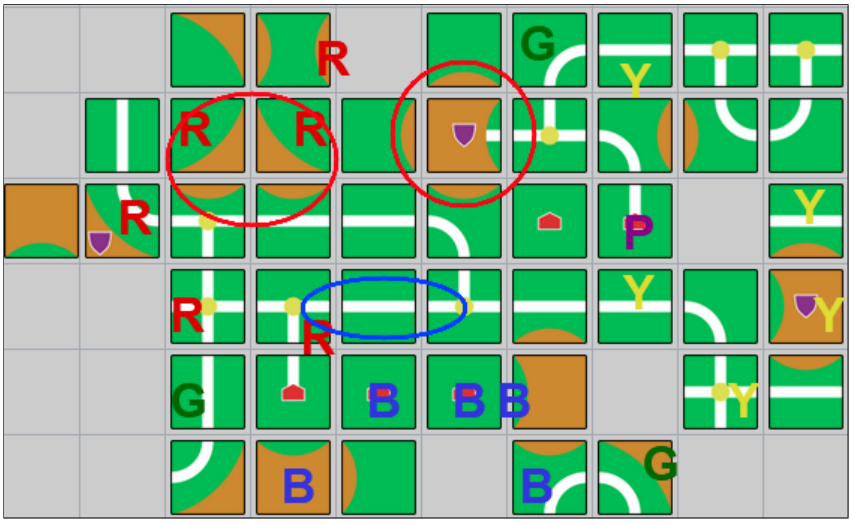



FIGURE 4 – Exemple de terrains complétés, avec deux *City* entourées en rouge et une *Road* entourée en bleu

### 1.3 Complexité

Bien que les règles de **Carcassonne** soient relativement simples, le jeu présente une richesse stratégique importante qui augmente à mesure que le plateau s'étend. Les choix offerts aux joueurs se multiplient progressivement en fonction de la configuration du plateau, des tuiles restantes et du placement des pions.

Contrairement à des jeux déterministes comme les échecs ou le shogi, **Carcassonne** introduit une composante stochastique via la pioche des tuiles, ce qui complexifie l'anticipation des coups futurs. Néanmoins, il demeure un jeu à information parfaite dans la mesure où l'ensemble des éléments placés sur le plateau est visible par tous les joueurs à tout instant.

La complexité combinatoire du jeu est particulièrement élevée en raison du nombre important de tuiles, de leurs différentes orientations possibles et des multiples placements légaux sur le plateau. La taille de l'arbre de jeu est estimée à environ $10^{194}$[5], ce qui rend toute exploration exhaustive irréaliste. Par conséquent, la résolution complète du jeu par des méthodes de recherche brute-force n'est pas envisageable.

Dans ce contexte, l'utilisation d'algorithmes de recherche heuristique et d'approximation apparaît comme une solution nécessaire pour proposer des décisions pertinentes dans un temps de calcul raisonnable.

## 2 Algorithmes et structures de données

Dans le cadre de ce projet, **Carcassonne** est considéré sous une forme restreinte pour la partie de réalisation d'intelligence artificielle afin de rendre possible l'utilisation d'algorithmes classiques de recherche dans les jeux.

Bien que **Carcassonne** ne soit pas strictement un jeu à somme nulle et qu'il comporte une composante stochastique liée à la pioche des tuiles, certaines hypothèses simplificatrices sont adoptées. En particulier, les décisions sont évaluées à partir de fonctions heuristiques estimant l'avantage relatif d'un joueur par rapport à son adversaire à un instant donné.

Cette modélisation permet l'utilisation d'algorithmes de recherche tels que Minimax et ses variantes, tout en restant consciente des limites théoriques de leur application à un jeu de cette nature.

Plusieurs algorithmes jouent ainsi un rôle central dans la réalisation des besoins du projet, notamment :

- **Minimax**[4], pour déterminer les coups optimaux en modélisant l'opposition entre les joueurs ;
- **Alpha-Beta**, qui permet de réduire le nombre d'états explorés en élaguant les branches non pertinentes de l'arbre de jeu ;
- **Iterative Deepening**, utilisé afin de borner le temps de calcul tout en améliorant progressivement la qualité des décisions ;
- **des fonctions d'évaluation heuristiques**, servant à estimer la qualité d'un état de jeu ;
- des approches basées sur *Monte Carlo Tree Search*[2], envisagées pour des stratégies alternatives ;
- des algorithmes de génération aléatoire, tels que le mélange de *Fisher–Yates*, nécessaires à la gestion équitable de la pioche.

La mise en œuvre efficace de ces algorithmes repose sur des structures de données adaptées permettant de représenter l'état du jeu et d'en faciliter la manipulation. Le plateau de jeu, constitué de tuiles connectées par leurs côtés, peut être modélisé à l'aide de graphes, offrant une représentation naturelle des relations d'adjacence entre les tuiles. Egalement, des piles peuvent être employées pour gérer certains mécanismes séquentiels, tels que la pioche des tuiles ou la sauvegarde temporaire d'états lors de l'exploration de l'arbre de jeu.

Dans la suite de cette section, nous nous concentrerons plus particulièrement sur les quatre premiers algorithmes présentés qui constituent le cœur du système de prise de décision implémenté dans le projet.

## 2.1 Algorithme Minimax

L'algorithme Minimax a pour but de déterminer les coups les plus optimaux pour un joueur à un état donné du jeu face à l'adversaire qui minimise le score du joueur "max". Ainsi, l'algorithme parcourt récursivement une partie de l'arbre des états du jeu, limitée par une profondeur maximale fixée afin de contenir la complexité de recherche. Chaque score d'une feuille est alors calculé via une fonction d'évaluation prédéfinie. Ce score remonte à son nœud parent qui sélectionne la valeur maximale ou minimale de ses nœuds enfants selon le joueur qu'il représente, et ainsi de suite jusqu'à la racine. L'algorithme possède 3 paramètres représentant l'état du jeu courant, la profondeur restante à explorer par rapport à l'état actuel, et un booléen à vrai lors du tour du joueur "max" et à faux sinon.

Voici le pseudocode de l'algorithme Minimax :

```
fonction minimax(etat, profondeur, joueur_max):
    SI profondeur = 0 OU estFeuille(etat) ALORS
        RETOURNER evalueHeuristique(joueur_max, etat)
    FIN SI
    SI joueur_max ALORS
        valeur = -infini
        POUR TOUT successeur s de etat FAIRE
            valeur = max(valeur, minimax(s, profondeur - 1, FAUX))
        FIN POUR TOUT
    FIN SI
    SINON
        valeur = +infini
        POUR TOUT successeur s de etat FAIRE
            valeur = min(valeur, minimax(s, profondeur - 1, VRAI))
        FIN POUR TOUT
    FIN SINON
    RETOURNER valeur
```

```
FIN FONCTION
```

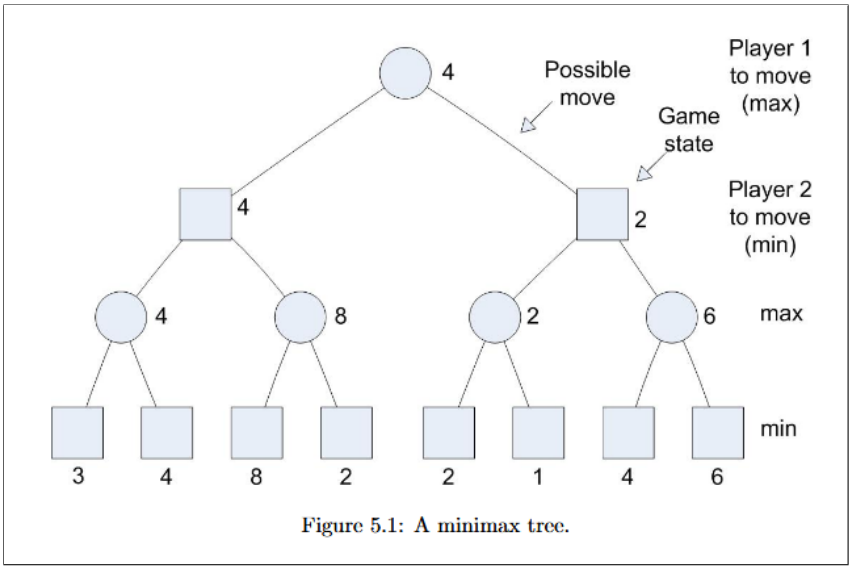


Figure 5.1: A minimax tree.

FIGURE 5 – Exemple d'un arbre de jeu partiel exploré par l'algorithme Minimax[5]

À la profondeur maximale à explorer, les feuilles remontent la valeur 4 (et non la valeur 3) lors du tour du joueur "max" pour maximiser son gain. Ainsi, au tour suivant, le joueur "min" récupère cette valeur 4 (et non la valeur 8) afin de minimiser le gain de son adversaire. Enfin, le joueur "max" récupère la valeur 4 (et non la valeur 2) pour maximiser son gain minimal. Il finit alors la recherche à la racine de l'arbre, ayant désormais connaissance d'une succession de coups optimaux à jouer pour obtenir un score de 4 en sa faveur à cette profondeur demandée.

## 2.2 Technique Iterative Deepening

L'algorithme Minimax avec approfondissement itératif (Iterative Deepening) est une technique de parcours progressif des profondeurs sur l'arbre des états d'un jeu. Après un parcours avec une nouvelle profondeur explorée, le meilleur coup pour le joueur "max" est enregistré en remplaçant le précédent, et sera alors renvoyé après épuisement du temps donné. Cette technique a pour avantage de déterminer une solution optimale à une profondeur quelconque même sur une courte durée, tandis que l'exploration classique ne garantit pas que la profondeur fixée soit suffisante pour obtenir un résultat fiable après une durée indéterminée.

## 2.3 Algorithme Alpha-Beta

Alpha-Beta (ou $\alpha\beta$-pruning) est une variante de l'algorithme Minimax consistant à explorer partiellement l'arbre des états d'un jeu en élaguant les branches qui n'apportent aucun gain nouveau pour le joueur "max". En stockant la meilleure valeur courante pour le joueur maximal ($\alpha$) et celle pour le joueur "min" ($\beta$) lors du parcours, l'algorithme n'évalue plus certains nœuds non-significatifs (et leurs potentiels enfants) quand $\alpha \geq \beta$, en réduisant potentiellement son temps d'exécution selon l'arbre donné.

## 2.4 Fonction d'évaluation heuristique

Étant donné l'impossibilité d'explorer l'arbre de jeu jusqu'à des états terminaux, l'algorithme Minimax repose sur une fonction d'évaluation heuristique permettant d'estimer la qualité d'un état de jeu intermédiaire. Cette fonction associe une valeur numérique à un état donné, représentant l'avantage relatif du joueur courant par rapport à son adversaire.

L'évaluation repose notamment sur les critères suivants :

- le score actuel des joueurs ;
- le potentiel de complétion des routes, cités et monastères en cours ;
- le nombre de pions disponibles pour chaque joueur ;
- la présence de pions bloqués sur des structures difficilement complétables ;
- le contrôle des champs adjacents à des cités complètes.

Ces différents éléments sont pondérés afin de refléter leur importance stratégique au cours de la partie. La fonction d'évaluation ne prétend pas fournir une estimation parfaite, mais permet de guider efficacement la recherche vers des coups jugés prometteurs.

## 2.5 Lien avec le projet

Les différentes techniques sont combinées dans le projet afin de rendre la recherche de coups pertinents possible malgré la complexité combinatoire élevée de Carcassonne. L'approche adoptée, mêlant Minimax, élagage Alpha-Beta, approfondissement itératif et fonctions d'évaluation heuristique, permet de guider efficacement la prise de décision des intelligences artificielles tout en respectant des contraintes de temps réalistes. La modélisation du plateau sous forme de graphes, associée à l'utilisation de structures de données adaptées comme les tables de hachage et les piles, assure une manipulation efficace de l'état du jeu et des configurations intermédiaires.

Ainsi, bien que l'exploration exhaustive de l'arbre de jeu reste impossible, le système mis en place fournit des conseils et des stratégies de qualité, exploitant de manière pragmatique les ressources computationnelles disponibles. Cette combinaison d'algorithmes et de structures de données constitue le cœur de l'intelligence artificielle du projet et offre une base solide pour l'extension future des fonctionnalités, telles que l'intégration de stratégies plus sophistiquées ou de variantes du jeu.

# 3 Architecture

Dans cette section, nous présentons l'architecture logicielle retenue pour l'implémentation du jeu Carcassonne. Le projet adopte une architecture 3-tiers[1] visant à séparer clairement les responsabilités afin d'améliorer la maintenabilité, la testabilité et l'évolutivité du code. Cette architecture nous paraît pertinente compte tenu du support naturel de la programmation orientée objet en Python. De plus, le projet amène naturellement à une structuration de ce genre. Cette architecture est structurée autour de trois modules principaux : Présentation, Logique Métier et Données, organisés selon des dépendances unidirectionnelles.

## 3.1 Découpage en modules

### Module Présentation

Le module Présentation est responsable des interactions avec l'utilisateur et de l'affichage de l'état du jeu. Il comprend notamment la gestion des commandes à travers le shell en CLI ou des actions à partir de la GUI, l'affichage du plateau, des tuiles, des meeples et des scores à travers CLI ou GUI.
Ce module ne contient aucune règle métier. Il délègue toute décision concernant la validité des actions ou l'évolution du jeu au module Logique Métier.

**Responsabilités principales :**

- Interprétation des actions utilisateur
- Appels aux interfaces de la logique métier
- Rafraîchissement de l'affichage

---

### Module Logique Métier

Le module Logique Métier constitue le cœur du jeu. Il encapsule l'ensemble des règles du jeu et gère l'état interne de la partie. Il est notamment responsable de la mise à jour du plateau (via une liste d'adjacence), du calcul des scores, de la gestion des tours de jeu et des conditions de fin de partie. Il prend également en charge les actions des intelligences artificielles, à l'aide de fonctions d'évaluation, ainsi que la gestion de l'état du serveur et des parties en cours.
Ce module est totalement indépendant de l'interface graphique et de la persistance.

**Responsabilités principales :**

- Implémentation des règles du jeu
- Gestion de l'état de la partie
- Gestion des IA de jeu
- Gestion des actions liées au serveur

---

### Module Données

Le module Données est dédié à la persistance et à la gestion des informations liées au jeu. Il s'occupe de la sauvegarde et du chargement des parties, de l'enregistrement de l'historique de jeu, ainsi que de la conservation des données relatives aux intelligences artificielles, notamment les structures de données et les meilleurs coups mémorisés dans le cadre de l'iterative deepening. Il gère également la persistance des parties en cours, de l'état du serveur et des informations associées aux joueurs.
Ce module n'implémente aucune règle métier et n'interagit jamais directement avec l'interface utilisateur.

**Responsabilités principales :**

- Sauvegarde des informations liées au jeu de près ou de loin (avec IA et serveurs) ;
- Possibilité de sauvegarder les parties en cours pour les recharger (dans des fichiers au format répondant aux fonctionnalités).

## 3.2 Dépendances entre modules

Les dépendances entre modules sont strictement unidirectionnelles (figure 6), conformément au principe d'architecture N-tiers.

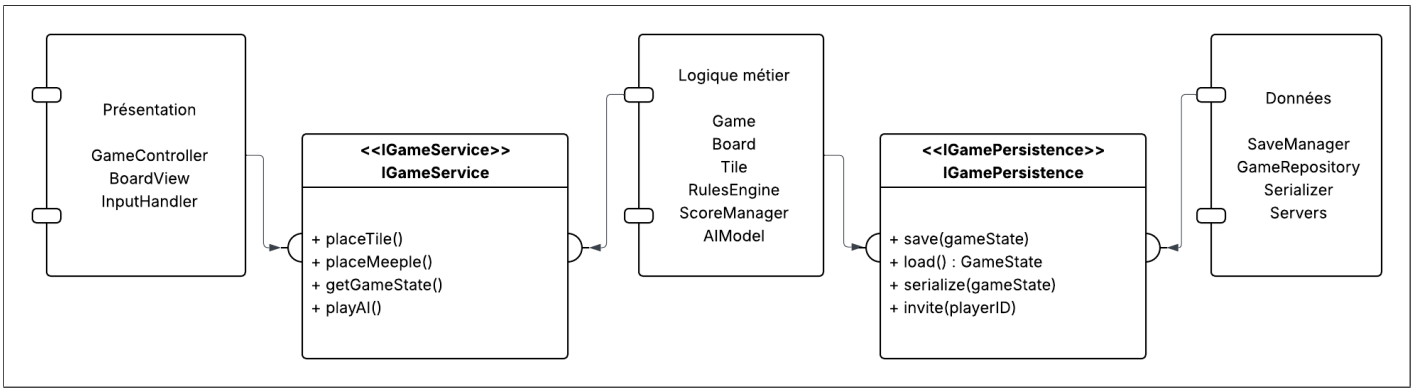



FIGURE 6 – Diagramme UML du projet

Ainsi, le module Présentation dépend uniquement du module Logique Métier, qu'il sollicite via des interfaces dédiées pour afficher l'état du jeu et transmettre les actions de l'utilisateur, sans accès direct aux données. Le module Logique Métier dépend uniquement du module Données pour la persistance des informations, cette interaction étant réalisée au moyen d'interfaces de persistance abstraites. Aucune dépendance directe n'existe entre la Présentation et les Données.

Cette organisation garantit une séparation claire des responsabilités et limite le couplage entre les composants.

## 3.3 Interfaces principales

Les interactions entre modules sont réalisées exclusivement via des interfaces bien définies.

Interface Présentation → Logique Métier

Exemples de services exposés :

```
place_tile(tile, position)
place_meeple(position)
get_game_state()
is_game_over()
```

Ces interfaces permettent à la présentation d'agir sur le jeu sans connaître l'implémentation interne des règles.

Interface Logique Métier → Données

Exemples de services exposés :

```
save_game(game_state)
load_game()
```

La logique métier délègue la persistance au module Données via ces interfaces abstraites.

# 4 Spécifications étendues

Dans cette section, nous détaillons les spécifications étendues du besoin fonctionnel **F.14 – Gestion des conseils**, dont l'objectif est de fournir au joueur une aide à la décision sous la forme d'un conseil de coup pertinent, tout en respectant des contraintes de validité et de performance.

## 4.1 Prérequis

Pour réaliser cette fonctionnalité, nous avons besoin de finir au préalable un grand nombre de besoins :

- F1 : Lancement du programme
- F4 : Interface en ligne de commande
- F11 : Gestion des règles
- F15 : Shell interactif
- F26 : Module *graph*
- F27 : Algorithmes de manipulation des graphes
- F32 : Minimax avec $\alpha\beta$-pruning
- F33 : Fonctions d'évaluation pour Minimax
- F34 : Approfondissement itératif de Minimax
- F35 : Profondeur de recherche de Minimax

## 4.2 Explication des sous-besoins

F1, F4 et F15 assurent un shell interactif dans lequel visualiser l'état de jeu et pouvoir demander un conseil au programme. F11 assure que les règles sont bien décrites et qu'on a une définition de coup légal. F26 et F27 permettent de stocker l'état de jeu dans la structure 'graph' et de pouvoir agir dessus avec les algorithmes pour notamment vérifier si le coup sur lequel se base le conseil est valide. F32, F33 et F35 apportent les algorithmes de parcours de l'arbre de jeu avec les fonctions d'évaluations pour décider du meilleur coup, on veut aussi avoir le moyen de définir une profondeur maximale afin de borner le temps de réponse. Pour ce faire, nous utilisons F34 qui permet d'évoluer profondeur par profondeur en stockant le meilleur résultat à chaque profondeur durant le parcours de l'arbre. Ainsi quand le temps de réponse arrive à échéance, on a un résultat optimal pour la profondeur atteinte prêt à être renvoyé.

## 4.3 Dépendance

Les sous-besoins engendrés par le besoin F14 se décomposent ainsi :

- **Fonctionnel :** créer une fonction permettant de répondre à la commande F15 "hint", avec une traduction compréhensible et simple du meilleur coup en conseil.
- **Non fonctionnel :** le temps de réponse de la commande doit être borné afin d'assurer une réponse rapide à l'utilisateur, sans bloquer l'exécution du jeu ou les autres joueurs de la partie. Pour garantir cette contrainte, une profondeur maximale de parcours de l'arbre peut être définie. L'utilisation de l'approfondissement itératif (F34) permet ainsi d'obtenir un résultat exploitable même en cas de contrainte temporelle stricte.

## 4.4 Tests unitaires

Cela engendre alors la création de la fonction suivante :

```
def give_hint(board: Board) -> None :
    """Affiche un conseil de coup à jouer dans le shell."""
```

Elle prend en argument l'état de jeu actuel et affiche dans le shell de l'exécutable le conseil. Cette fonction est accompagnée d'un test unitaire vérifiant bien que la commande écrive quelque chose dans le shell. On peut imaginer en entrée du test un état de jeu avec la prochaine tuile à jouer qu'on lui impose, et vérifier que la fonction s'exécute sans erreur et produit une sortie non vide. On a aussi un test unitaire pour vérifier la rapidité du temps de réponse, en donnant en entrée un état de jeu avec une tuile à jouer et en vérifiant que le temps de réponse de la fonction *give_hint* n'excède pas une limite raisonnable qu'on se fixe.

## 4.5 Intégration dans le projet

La fonction *give_hint* serait présente dans le module Logique Métier de notre architecture, elle serait appelée par la commande hint du module Présentation, et *give_hint* irait chercher l'état de jeu actuel dans le module Données afin de pouvoir se réaliser.

## 4.6 Validation

Pour valider ce besoin, il faudrait que tous les besoins prérequis le soient, et que les tests associés aux sous-besoins passent. Pour assurer que la fonctionnalité est implémentée, on peut faire un test en partant d'une partie avec un état donné et regarder si l'appel à *give_hint* produit bien un conseil dans le shell. Pour assurer que le conseil se base sur un coup valide, on peut regarder si ce dernier est en effet un coup possible depuis l'état de jeu. Pour assurer que le conseil fourni est pertinent, on peut le comparer au conseil optimal à donner dans cette situation. Et pour tester le temps de réponse, on peut partir d'un état de jeu avancé à partir duquel le facteur de branchement est important pour assurer que le conseil donné est bien délivré dans le temps imparti.

# Références


[1] Nawar S Alseelawi, Enas K Adnan, Hussein T Hazim, Haider Alrikabi, and Khalid Nasser. Design and implementation of an e-learning platform using n-tier architecture. *International Journal of Interactive Mobile Technologies (iJIM)*, 2020.

[2] Fred Valdez Ameneyro, Edgar Galván, and Ángel Fernando Kuri Morales. Playing carcassonne with monte carlo tree search. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 2343–2350. IEEE, 2020.

[3] Christopher Obe. Carcassonne complete annotated rules. `https://www.dover.nh.gov/Assets/government/city-operations/library/borrow/library-of-things/carcassonne-rules.pdf`, 2014.

[4] Thomas Hauk, Michael Buro, and Jonathan Schaeffer. Rediscovering*-minimax search. In *International Conference on Computers and Games*, pages 35–50. Springer, 2004.

[5] Cathleen Heyden. Implementing a computer player for carcassonne. *Master's thesis, Department of Knowledge Engineering, Maastricht University*, 2009.